LAPORAN PRAKTIKUM — JS-DVO-02 (Minggu 2)

Mata Kuliah : DevOps (RKS305) · **Program Studi** : Rekayasa Keamanan Siber

Mahasiswa : Muhammad Gandhi Putra (NIM 4332511050)

Tanggal praktikum : 22 September – 28 September 2026

Repo GitHub : https://github.com/GJRRR/praktikum-devops-m01

Topik : Linux Foundation for DevOps — Shell Scripting, CLI, & Manajemen Sistem

---

1. Pendahuluan & Ringkasan Pengerjaan

Praktikum minggu ke-2 menjembatani budaya DevOps dengan fondasi Linux operasional. Seluruh pekerjaan diotomasi melalui antarmuka baris perintah (CLI) dan skrip Bash yang **lolos** `shellcheck` **tanpa satu pun peringatan** (prasyarat mutlak rubrik JOB 4, bobot 25%).

Artefak yang dihasilkan:

Job	Skrip / Artefak	Status shellcheck
JOB 1	(perintah CLI FHS, chmod, umask, find)	-
JOB 2	`buat-log.sh`, `app.log`, `ringkasan-log.txt`	CLEAN
JOB 3	(ps/top/ss/free/systemctl, SIGTERM)	-
JOB 4	`sysreport.sh`	CLEAN
JOB 5	`lib/common.sh`, `setup.sh` (refaktor M01)	CLEAN
Challenge	`healthwatch.sh`	CLEAN

---

2. Bukti Penggunaan (Lampiran Wajib)

2.1 Tautan Repository

https://github.com/GJRRR/praktikum-devops-m01

2.2 Bukti shellcheck Bersih

Perintah yang dijalankan di mesin (Ubuntu 22.04 CLI):

```
shellcheck buat-log.sh sysreport.sh healthwatch.sh \
../minggu-01/app-sentra/lib/common.sh \ ../minggu-01/app-sentra/setup.sh
```

Keluaran: **(kosong)** — tidak ada peringatan maupun galat. Artinya kelima skrip memenuhi syarat "lolos shellcheck tanpa peringatan sama sekali". (Tangkapan layar `shellcheck` tanpa output disertakan terpisah pada berkas lampiran praktikan.)

2.3 Tabel Exit Code `sysreport.sh` (Hasil Pengujian)

Perintah	Exit code	Arti
`./sysreport.sh`	0	sehat (disk < ambang)
`./sysreport.sh -j`	0	sehat, keluaran JSON
`./sysreport.sh -d 1`	2	melewati ambang (disk >= 1%) → status PERINGATAN
`./sysreport.sh -d abc`	1	galat penggunaan (validasi angka gagal)

`./sysreport.sh -z`	1	opsi tidak dikenal
`./sysreport.sh -h`	0	tampilkan bantuan

Pembedaan 0 / 1 / 2 bersifat krusial: runner CI/CD (GitHub Actions / GitLab CI) **hanya membaca exit code**, bukan tampilan. Kode 2 = "sistem tidak sehat namun skrip jalan" harus dapat dibedakan dari 1 = "pemanggilan salah", agar insiden nyata tidak tertelan atau malah memicu *alert fatigue*.

2.4 Cuplikan `/tmp/sentra-deploy.log` (JOB 5)

```
2026-09-28T13:00:00+0700 [INFO ] [1/5] Memeriksa prasyarat... 2026-09-28T13:00:00+0700 [INFO ] Python: Python 3.10.12 2026-09-28T13:00:01+0700 [INFO ] [2/5] Menentukan port... 2026-09-28T13:00:01+0700 [INFO ] Port 5000 tersedia. 2026-09-28T13:00:01+0700 [INFO ] [3/5] Menyiapkan virtual environment... 2026-09-28T13:00:01+0700 [INFO ] venv sudah ada, digunakan kembali (idempotent). 2026-09-28T13:00:02+0700 [INFO ] [4/5] Memasang dependensi terkunci... 2026-09-28T13:00:05+0700 [INFO ] [5/5] Menjalankan aplikasi pada port 5000... 2026-09-28T13:00:07+0700 [INFO ] SUKSES: aplikasi sehat pada percobaan ke-1.
```

Versi modular menulis log berstempel ISO-8601 + level (`[INFO]/[WARN]/[ERROR]`) ke `/tmp/sentra-deploy.log` secara *append*. Informasi baru yang didapat: waktu pasti tiap langkah dan tingkat keparahan, yang mempercepat *root-cause analysis* secara drastis bila terjadi insiden.

2.5 Berkas `ringkasan-log.txt` (JOB 2)

Berkas terlampir (`minggu-02/ringkasan-log.txt`). Ringkasannya:

```
Total permintaan : 500 200:200 404:105 500:55 201:52 301:46 502:42 5 klien paling aktif: 192.168.5.20(105) 172.16.4.7(102) 10.10.0.13(100) ... Endpoint gagal (>=500): /static/app.js 21, / 21, /health 20, /api/orders 20, /api/users 15
```

---

3. Pertanyaan Analisis (4.1)

3.1 Batas Kemampuan `set -e`

Skrip kami memuat `set -euo pipefail`, namun `errexit` **tidak** menghentikan eksekusi pada semua kondisi. Minimal dua situasi di mana perintah gagal namun skrip tetap jalan:

1. **Di dalam kondisi `if` / `while` / `until`.** `set -e` sengaja dimatikan untuk perintah yang menjadi bagian dari evaluasi kondisi. Contoh:

```
if grep "pola" berkas.txt; then echo "ketemu"; fi
```

`grep` mengembalikan *exit 1* bila pola tidak ditemukan, tetapi skrip **tidak** berhenti — kegagalan itu justru merupakan hasil yang diuji oleh `if`. Demikian pula `while read -r baris; do ...; done < file`.

2. **Di sisi kiri operator `||` (OR).** Pada `cmd1 || cmd2`, kegagalan `cmd1` adalah hasil yang diharapkan (fallback ke `cmd2`), sehingga `errexit` tidak memicu. Pola `mkdir dir || true` pun sengaja membutakan pipeline — sebuah *anti-pattern* di CI karena menyembunyikan kegagalan.

3. *(Tambahan)* **Command substitution.** Pada `var=$(cmd_yang_gagal)`, kegagalan di dalam `$(...)` **tidak** memicu `set -e`; status keluar substitution itu sendiri terbuang begitu saja.

Penanganan yang tepat:

- Tambahkan `set -E` (*errtrace*) agar `trap ... ERR` ikut aktif di dalam fungsi.
- Untuk *command substitution*, periksa secara eksplisit:

```
var=$(cmd) || die "cmd gagal" # atau if ! var=$(cmd); then die "cmd gagal"; fi
```

- Jangan mengandalkan `set -e` sebagai pengganti validasi; selalu periksa *return code* perintah kritis (terutama di ujung pipeline, ditambah `pipefail` yang sudah kami gunakan).
- Hindari pola `|| true` yang sengaja menutupi kegagalan dalam alur CI.

3.2 Agregat yang Menyesatkan

Proksi Change Failure Rate (CFR) di sini diartikan sebagai proporsi respons 5xx (kegagalan server) terhadap total permintaan:

```
CFR = (500×55 + 502×42) / 500 = 97 / 500 = 19,40 %
```

(Kode 4xx seperti 404 tidak dihitung karena merupakan *client error*, bukan kegagalan server.)

Mengapa satu angka agregat dapat menyesatkan tim operasional — merujuk sebaran kegagalan per endpoint:

Endpoint	Kegagalan >=500
/static/app.js	21
/	21
/health	20
/api/orders	20
/api/users	15

- Kegagalan **tersebar merata** di **seluruh** endpoint, bukan terpusat di satu rute. Angka 19,40% tampak "masuk akal" dan bisa dianggap wajar, padahal akar masalahnya **sistemik** (dependensi bersama, kapasitas backend, atau satu *deploy* buruk yang menimpa semua rute). Tim yang hanya melihat CFR agregat akan salah sasaran memperbaiki satu endpoint — padahal memperbaikinya tidak menurunkan CFR.
- CFR di **level proxy** menyembunyikan *service/upstream* mana yang gagal; proxy hanya melihat status akhir, bukan penyebab hulu.
- Agregat menyatukan **latensi ekor (p95/p99)** dan **SLO per-endpoint**; satu endpoint kritis dapat melanggar SLO sementara rata-rata masih hijau.

Metrik tambahan yang lebih berguna untuk prioritas perbaikan:

- **Error rate per-endpoint** (5xx per endpoint / total per endpoint) — langsung menunjukkan rute bermasalah.
- **Error budget / burn-rate** berbasis SLO per layanan (multi-window burn-rate alerting ala Google SRE).
- **CFR per upstream/service**, bukan per proxy, ditambah *distributed tracing* (traceID) untuk korelasi hulu-hilir.
- **Korelasi dengan garis waktu deploy** — apakah CFR naik setelah *deploy* tertentu?

3.3 CLI versus GUI

Antarmuka baris perintah menjadi prasyarat otomasi karena tiga sifat:

1. **Reproducibility (reproduksibilitas).** Skrip CLI deterministik: input sama menghasilkan urutan langkah yang sama, tanpa variasi klik manusia. GUI rentan terhadap langkah terlewat atau urutan berbeda antar-operator.

2. **Auditability (ketertelusuran).** Seluruh tindakan CLI terekam sebagai teks (log, *history*); kita dapat memeriksa persis apa yang berjalan. GUI tidak meninggalkan jejak terstruktur yang mudah diaudit.

3. **Version control.** Skrip dan konfigurasi CLI berupa teks → dapat masuk `git`, di-*review* via *pull request*, di-*diff*, dan di-*rollback*. GUI tidak dapat disimpan ke VCS.

Situasi konkret di mana GUI tetap lebih tepat: eksplorasi data visual / *adhoc* (mis. membaca anomali di dashboard Grafana, memahami *topology map* jaringan, atau pengolahan citra/spasial). Untuk tugas satu kali (*one-off*) yang tidak perlu direproduksi, GUI jauh lebih cepat daripada menulis skrip. Contoh: mengidentifikasi penyebab *spike* latency lewat grafik interaktif lebih efisien daripada menulis pipeline `awk` dari awal.

3.4 Least Privilege pada Otomasi

Saran rekan: jalankan `setup.sh` dengan `sudo` dan beri izin `777` pada seluruh direktori aplikasi agar tidak pernah muncul galat *Permission denied*.

Risiko konkret:

- `chmod 777` = `rwxrwxrwx` → **semua** pengguna & proses lokal dapat membaca rahasia **dan** menulis/mengganti biner serta skrip aplikasi. Penyerang (atau malware) dapat menyuntik kode yang kemudian dieksekusi layanan → *privilege escalation*.
- `sudo setup.sh` → **seluruh** skrip (termasuk *bug* atau dependensi yang di-*source*) berjalan sebagai *root*; *blast radius* = sistem penuh. Satu *typo* atau dependensi terkompromi = kompromi *root*.
- Keduanya melanggar prinsip *least privilege* sepenuhnya.

Dua mekanisme pengganti yang tetap menyelesaikan masalah tanpa menaikkan hak:

1. **Akun layanan khusus + sudoers allowlist sempit.** Jalankan sebagai user `sentra` (pemilik direktori aplikasi & venv). Hanya langkah istimewa (mis. `systemctl restart sentra`) diberi izin via `/etc/sudoers`:

```
sentra ALL=(root) NOPASSWD: /usr/bin/systemctl restart sentra
```

Hanya perintah itu yang diizinkan sebagai *root* — bukan keseluruhan skrip.

2. **Rootless / capability terbatas.** Gunakan *rootless container* (Podman) atau systemd `User=sentra` + `CapabilityBoundingSet=CAP_NET_BIND_SERVICE` agar aplikasi dapat *bind* port <1024 tanpa *root* penuh; alternatifnya `setcap cap_net_bind_service=ep ./app` untuk izin minimal. Solusi ini menghilangkan *Permission denied* dengan memberi hak sesempit mungkin, bukan 777 + sudo.

3.5 Idempotensi

Uji: jalankan `sysreport.sh` dan `setup.sh` masing-masing **tiga kali berturut-turut**.

- `sysreport.sh` → **IDEMPOTEN.** Skrip *read-only*, tidak mengubah *state* sistem (hanya membaca penggunaan disk, memori, dan jumlah proses). Eksekusi ke-1, ke-2, ke-3 menghasilkan laporan yang sama tanpa menciptakan berkas/proses baru. Bukti: 3× menjalankan tidak meninggalkan *side effect* yang menumpuk.
- `setup.sh` → **TIDAK sepenuhnya idempoten pada alur utama.** Langkah provisi memang idempoten: `check_prereqs` hanya memeriksa; `prepare_venv` memiliki *guard* `[ -d "$VENV_DIR" ]` sehingga venv dipakai kembali; `install_deps` via `pip` bersifat *no-op* bila sudah terpasang. **Namun** `run_app` **meluncurkan proses baru tiap pemanggilan:**

```
: > "$APP_DIR/app.log" PORT=... "$PY" src/app.py >> app.log 2>&1 &
```

Jika *instance* sebelumnya masih memegang port 5000, run ke-2 mendeteksi port penuh lalu `find_free_port` memindah ke 5001 dan **meluncurkan instance kedua**, bukan menjadi *no-op*. Dengan demikian pengulangan penuh **menumpuk proses** (5000/5001/5002), bukan konvergen ke satu *state* stabil.

- Bukti: 3× `./setup.sh` tanpa `--stop` di antaranya → 3 proses Flask pada port berbeda. Mode `--check` sendiri idempoten & aman.
- **Perbaikan agar** `setup.sh` **penuh idempoten:** tambahkan *guard* di `run_app` — bila `port_is_free` bernilai *false* **dan** endpoint `/health` merespons, anggap layanan sudah jalan dan `exit 0` (no-op); bila port penuh namun tidak sehat, panggil `stop_app` lalu lanjut. Maka *re-run* = *state* konvergen.

Kaitan dengan Infrastructure as Code (Minggu 12): IaC mengeksekusi *playbook/manifest* berulang kali (CI tiap *commit*, atau *self-healing* otomatis). Bila skrip tidak idempoten, eksekusi ke-2 merusak *state* (membuat duplikat, gagal, atau *corrupt*). Idempotensi menjamin "deklarasi = realitas" terwujud berulang kali tanpa efek samping menumpuk — syarat mutlak agar Terraform/Ansible/ArgoCD dapat dijalankan aman berkali-kali.

---

4. Tantangan: Health Watcher (`healthwatch.sh`)

Skrip `healthwatch.sh` memantau endpoint kesehatan aplikasi Sentra dengan spesifikasi berikut:

- **Argumen:** `--interval N` (detik, baku 5), `--url URL` (baku `http://127.0.0.1:5000/health`), `--log BERKAS` (baku `healthwatch.log`), `-h`. Seluruh argumen divalidasi: `--interval` harus bilangan bulat >= 1, `--url` harus diawali `http:///https://`; nilai tidak masuk akal ditolak dengan *exit 1*.
- **Log terstruktur:** satu baris per pemeriksaan berisi stempel ISO-8601, status `UP`/`DOWN`, dan waktu respons (ms).
- **Deteksi transisi:** saat `UP -> DOWN` dicatat waktu mulai insiden; saat `DOWN -> UP` dihitung **MTTR** (Mean Time to Restore) untuk insiden tersebut.
- `trap` **SIGINT:** `Ctrl+C` mencetak ringkasan sesi — jumlah pemeriksaan, jumlah insiden, total waktu padam, dan ketersediaan (%).
- **Exit code bermakna:** `0` bila tanpa insiden, 2 bila terjadi minimal satu insiden.
- **shellcheck:** lolos tanpa peringatan (CLEAN).

Prosedur uji wajib (dijalankan di Ubuntu):

```
# Terminal 1: jalankan aplikasi Sentra (setup.sh dari minggu-01) ./minggu-01/app-sentra/setup.sh # Terminal 2: pantau ./healthwatch.sh --interval 5 # tunggu beberapa detik, lalu hentikan aplikasi: kill -TERM <PID_flask> # atau ./setup.sh --stop # tunggu ~15 detik (tercatat sebagai DOWN + insiden) # hidupkan kembali aplikasi, tunggu kembali UP (MTTR tercatat) # tekan Ctrl+C -> cetak ringkasan sesi
```

Contoh keluaran ringkasan sesi (diharapkan):

```
=== Ringkasan Sesi healthwatch === Jumlah pemeriksaan : 24 Jumlah insiden : 1 Total waktu padam : 15 detik Ketersediaan : 96.88 %
```

(Tangkapan layar ringkasan sesi disertakan terpisah pada lampiran praktikan.)

---

5. Kesimpulan

Minggu ke-2 mengukuhkan bahwa DevOps bukan sekadar alat, melainkan cara kerja berbasis teks, otomasi, dan prinsip keamanan. Tiga benang merah terpenting:

1. **CLI & skrip = fondasi otomasi** — reproducible, auditable, dan version-controlled, berbeda dengan GUI yang hanya cocok untuk eksplorasi adhoc.

2. **shellcheck & exit code bermakna** adalah kontrak dengan CI/CD; tanpa keduanya pipeline buta terhadap kegagalan.

3. **Least privilege & idempotensi** bukan teori, melainkan prasyarat agar otomasi aman dijalankan berulang kali (menuju IaC di minggu ke-12).

Analisis log JOB 2 membuktikan bahaya agregat tunggal: CFR proksi 19,40% menutupi fakta bahwa kegagalan menyebar sistemik ke seluruh endpoint — metrik per-endpoint dan error budget jauh lebih actionable.